Microservices Authentication & Authorization Architecture Guide

Enterprise Security Architecture for Real-Time Order Processing System

Version: 2.0
Date: March 2026
Classification: Internal – Architecture Reference
Author: Security Architecture Team

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Authentication Deep Dive
4. Authorization Deep Dive
5. JWT Deep Dive
6. Step-by-Step Implementation Guide
7. Code Walkthrough
8. Production-Grade Enhancements
9. Security Best Practices
10. Common Mistakes & Anti-Patterns
11. Interview Preparation Questions
12. Future Scalability Considerations

---

1. Executive Summary

This document provides a comprehensive, enterprise-level architecture guide for implementing Authentication and Authorization in a Spring Boot Microservices ecosystem. It is designed as a **build-from-scratch reference** — every component is explained with its **WHY** (business justification), **WHAT** (problem it solves), and **HOW** (internal working mechanism).

System Under Scope

The Real-Time Order Processing System comprises the following services:

Service	Port	Database	Security Role
Eureka Server	8761	None	Service Registry (no auth)
API Gateway	8080	None	JWT Validation, Route Security, CORS
Auth Service	8081	auth_db (MySQL)	JWT Generation, User Registration, Login
User Service	8082	user_db	Profile Management, RBAC Enforcement
Product Service	8083	product_db	Product CRUD, Admin Protection
Order Service	8084	order_db	Order Processing, Admin Protection
Inventory Service	8085	inventory_db	Stock Management, Fine-grained RBAC

Payment Service	8086	payment_db	Payment Processing, JWT Protected
Notification Service	8087	notification_db	Audit Logs, Admin Protection

Security Architecture Summary

- **Authentication Model:** Stateless JWT (JSON Web Token)
- **Authorization Model:** Role-Based Access Control (RBAC) with two roles: `USER` and `ADMIN`
- **Token Signing:** HMAC-SHA256 (Symmetric Key)
- **Password Hashing:** BCrypt (10 rounds)
- **Gateway Pattern:** Centralized JWT validation at API Gateway layer
- **Role Propagation:** Custom HTTP headers (`X-User-Name` , `X-User-Role`)
- **Session Management:** Fully stateless (`SessionCreationPolicy.STATELESS`)
- **CSRF Protection:** Disabled (justified by stateless token-based architecture)

Key Design Decisions

1. **Why JWT over Sessions?** — Horizontal scalability without shared session stores
2. **Why Gateway-level validation?** — Single security checkpoint; downstream services trust the Gateway
3. **Why Header-based role propagation?** — Eliminates JWT re-parsing overhead in every microservice
4. **Why BCrypt?** — Adaptive hashing with configurable cost factor resists GPU-based brute force
5. **Why Symmetric signing (HMAC)?** — Simpler key management for internal services; asymmetric recommended for third-party integrations

---

2. Architecture Overview

2.1 High-Level Security Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        INTERNET / CLIENT LAYER                           │
│                                                                          │
│    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐             │
│    │  React App   │    │  Mobile App  │    │  Postman     │             │
│    │  (Port 3000) │    │  (iOS/Android)│   │  (Testing)   │             │
│    └──────┬───────┘    └──────┬───────┘    └──────┬───────┘             │
│           └──────────────────┬──────────────────┘                       │
│                              │  HTTP + JWT Bearer Token                 │
└──────────────────────────────┼──────────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                        API GATEWAY (Port 8080)                           │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                     Security Filter Pipeline                     │   │
│  │                                                                  │   │
│  │  1. CorsWebFilter          → Handles CORS preflight (OPTIONS)    │   │
│  │  2. JwtAuthenticationFilter → Validates JWT, extracts claims     │   │
│  │  3. CircuitBreaker          → Resilience for downstream calls    │   │
│  │  4. Route Predicate         → Routes to correct microservice     │   │
│  │                                                                  │   │
│  │  OPEN ENDPOINTS (No JWT Required):                               │   │
```

```
│  │    • /api/auth/signup                                                  │ │
│  │    • /api/auth/login                                                   │ │
│  │    • /api/auth/validate                                                │ │
│  │    • /fallback/*                                                       │ │
│  │                                                                         │ │
│  │  PROTECTED ENDPOINTS (JWT Required):                                   │ │
│  │    • /api/users/**     → JwtAuthenticationFilter → User Service        │ │
│  │    • /api/products/**  → JwtAuthenticationFilter → Product Service     │ │
│  │    • /api/orders/**    → JwtAuthenticationFilter → Order Service        │ │
│  │    • /api/inventory/** → JwtAuthenticationFilter → Inventory Service   │ │
│  │    • /api/payments/**  → JwtAuthenticationFilter → Payment Service     │ │
│  │    • /admin/**         → JwtAuthenticationFilter → Respective Service  │ │
│  └──────────────────────────────────────────────────────────────────────┘ │
│                                                                             │
│  On successful JWT validation, Gateway adds headers:                        │
│    X-User-Name: <extracted_username>                                        │
│    X-User-Role: <extracted_role>                                            │
│                                                                             │
└─────────────┬──────────────┬──────────────┬──────────────┬──────────────┘
              │              │              │              │              │
              ▼              ▼              ▼              ▼              ▼

┌──────────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
│ Auth Service │ │   User   │ │ Product  │ │  Order   │ │ Payment  │
│ (Port 8081)  │ │ Service  │ │ Service  │ │ Service  │ │ Service  │
│              │ │(Port 8082)│ │(Port 8083)│ │(Port 8084)│ │(Port 8086)│
│ • SecurityConf │ • Internal │ • Internal │ • Internal │ • Internal │
│ • JwtUtil    │ │  AuthFilt │ │  AuthFilt │ │  AuthFilt │ │  AuthFilt │
│ • AuthService │ • @PreAuth │ • @PreAuth │ • @PreAuth │ • Security │
│ • BCrypt     │ • RBAC    │ • RBAC    │ • RBAC    │ │  Config  │
│ • No JWT check │          │          │          │ │          │
│  on /api/auth │ │          │ │          │ │          │ │          │
└──────────────┘ └──────────┘ └──────────┘ └──────────┘ └──────────┘
       │                                                       │
       ▼                                                       ▼
  ┌─────────┐                                            ┌──────────┐
  │ auth_db │          All services register with →      │ Eureka   │
  │ (MySQL) │                                            │ Server   │
  │         │                                            │(Port 8761)│
  └─────────┘                                            └──────────┘
```

2.2 Authentication Flow Diagram

```
┌─────────┐          ┌──────────┐          ┌──────────────┐          ┌─────────┐
│ Client  │          │API Gateway│          │ Auth Service │          │ auth_db │
└────┬────┘          └────┬─────┘          └────┬─────────┘          └────┬────┘
     │                    │                      │                         │
     │  1. POST /api/auth/login                  │                         │
     │  {username, password}                     │                         │
     │ ──────────────────────►                   │                         │
     │                    │                      │                         │
     │                    │  2. Route to Auth    │                         │
```

```
     │                    │  (No JWT filter for   │                    │
     │                    │   /api/auth/**)       │                    │
     │                    │ ────────────────────► │                    │
     │                    │                       │                    │
     │                    │                       │ 3. SELECT * FROM   │
     │                    │                       │    user_credentials │
     │                    │                       │    WHERE username=? │
     │                    │                       │ ─────────────────► │
     │                    │                       │                    │
     │                    │                       │ 4. Return user record│
     │                    │                       │ ◄───────────────── │
     │                    │                       │                    │
     │                    │                       │ 5. BCrypt.matches( │
     │                    │                       │    rawPwd, hashedPwd)│
     │                    │                       │                    │
     │                    │                       │ 6. If valid:       │
     │                    │                       │    Generate JWT with │
     │                    │                       │    {sub: username, │
     │                    │                       │     role: USER/ADMIN,│
     │                    │                       │     iat: now,      │
     │                    │                       │     exp: now+24h}  │
     │                    │                       │                    │
     │                    │  7. Return AuthResponse                    │
     │                    │  {token, username,     │                    │
     │                    │   role, message}       │                    │
     │                    │ ◄──────────────────── │                    │
     │                    │                       │                    │
     │  8. Return JWT token│                      │                    │
     │ ◄────────────────── │                      │                    │
     │                    │                       │                    │
     │  9. Client stores token in localStorage    │                    │
     │     or secure cookie                       │                    │
     │                    │                       │                    │
```

2.3 Authorization Flow Diagram

```
┌──────────┐        ┌──────────┐          ┌──────────────┐
│  Client  │        │API Gateway│          │Business Service│
│          │        │          │          │(e.g., Order)  │
└──────────┘        └──────────┘          └──────────────┘
     │                   │                        │
     │ 1. GET /admin/orders                       │
     │ Authorization: Bearer <JWT>                │
     │ ────────────────► │                        │
     │                   │                        │
     │                   │ 2. JwtAuthenticationFilter│
     │                   │    a. Extract token    │
     │                   │    b. Validate signature│
     │                   │    c. Check expiration │
     │                   │    d. Extract claims:  │
     │                   │       username="admin1"│
```

```
    |                    |       role="ADMIN"        |
    |                    |                           |
    |                    | 3. Mutate request:        |
    |                    |    Add X-User-Name=admin1 |
    |                    |    Add X-User-Role=ADMIN  |
    |                    | ──────────────────────────►|
    |                    |                           |
    |                    |                           | 4. InternalAuthFilter:
    |                    |                           |    Read X-User-Name
    |                    |                           |    Read X-User-Role
    |                    |                           |    Create Authentication:
    |                    |                           |    ROLE_ADMIN
    |                    |                           |    Set SecurityContext
    |                    |                           |
    |                    |                           | 5. SecurityFilterChain:
    |                    |                           |    /admin/** requires
    |                    |                           |    hasRole("ADMIN")
    |                    |                           |    ✓ ROLE_ADMIN matches
    |                    |                           |
    |                    |                           | 6. @PreAuthorize check:
    |                    |                           |    hasRole('ADMIN')
    |                    |                           |    ✓ Passes
    |                    |                           |
    |                    | 7. Response 200 OK        |
    | ◄──────────────────┼───────────────────────────|
    |                    |                           |
```

2.4 Stateless vs Stateful Security Comparison

Aspect	Stateful (Session-Based)	Stateless (JWT-Based) ✓ Used
Storage	Server-side session store (Redis, DB)	Client-side token (localStorage, cookie)
Scalability	Requires session replication/sticky sessions	Scales horizontally without shared state
Performance	DB/Redis lookup on every request	Cryptographic validation only (no I/O)
Microservices Fit	Poor — requires shared session store	Excellent — each service validates independently
Logout	Simple — destroy session	Complex — requires token blacklisting
Token Size	Small session ID (~32 bytes)	Larger JWT (~500+ bytes)
Security	Session fixation attacks possible	Token theft / replay attacks possible
CSRF	Vulnerable — requires CSRF tokens	Not vulnerable (when using Authorization header)
Network	Larger cookies on every request	Bearer token only on API calls

WHY JWT over Sessions in this project:

1. **Horizontal Scalability** — Multiple instances of each service can run behind a load balancer without session affinity
2. **Microservice Independence** — No shared session store needed between 8+ services
3. **API Gateway Pattern** — Gateway validates once, propagates identity via headers
4. **Mobile Client Support** — JWT works natively with REST APIs from any client
5. **Performance** — No database/Redis round-trip for session validation on every request

2.5 Why API Gateway Validates Tokens

```
WITHOUT GATEWAY VALIDATION (Anti-pattern):
──────────────────────────────────────────

Client → Order Service (validate JWT) → Process
Client → User Service (validate JWT) → Process
Client → Product Service (validate JWT) → Process
Client → Payment Service (validate JWT) → Process

PROBLEMS:
  • Every service needs JWT library + secret key
  • JWT parsing duplicated 6+ times
  • Secret key spread across all services
  • Inconsistent validation logic risks

WITH GATEWAY VALIDATION (Current Architecture):
───────────────────────────────────────────────

Client → API Gateway (validate JWT once) → Forward with headers → Service processes

BENEFITS:
  ✓ Single point of JWT validation
  ✓ Secret key only in Gateway + Auth Service
  ✓ Downstream services are simpler
  ✓ Consistent security enforcement
  ✓ Easy to add rate limiting, IP blocking at one point
```

---

3. Authentication Deep Dive

3.1 What is Authentication?

Authentication answers the question: **"Who are you?"** It is the process of verifying a user's identity by validating their credentials (username/password) against stored records.

3.2 Why JWT-Based Authentication?

In a monolithic application, a simple `HttpSession` would suffice. In microservices, we face these challenges:

Challenge	Session-Based Solution	JWT-Based Solution
Multiple service instances	Sticky sessions / Redis session store	No shared state needed
Cross-service identity	Session replication	Self-contained token carries identity
Mobile clients	Cookie limitations	Authorization header works everywhere

API-first design	Cookie-dependent	Bearer token is REST-native
Performance	Network I/O for session lookup	CPU-only cryptographic validation

3.3 Authentication Components in This Project

3.3.1 SecurityConfig (Auth Service)

WHY it exists: Configures Spring Security for the Auth Service — the only service that handles user registration and login.

WHAT problem it solves: Ensures that authentication endpoints (`/api/auth/**`) are publicly accessible while all other endpoints require authentication. Disables CSRF and sessions because the architecture is stateless.

HOW it works internally:

```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
                .csrf(csrf -> csrf.disable())                            // 1
                .sessionManagement(session -> session
                        .sessionCreationPolicy(SessionCreationPolicy.STATELESS))  // 2
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/**").permitAll()        // 3
                        .anyRequest().authenticated());                  // 4
        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();                              // 5
    }

    @Bean
    public AuthenticationManager authenticationManager(
            AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();                        // 6
    }
}
```

Line-by-line explanation:

1. **CSRF disabled** — CSRF attacks exploit session cookies. Since we use Bearer tokens (not cookies), CSRF is not a threat vector. The token must be explicitly included in the `Authorization` header, which JavaScript from another origin cannot do due to CORS.
2. **Stateless sessions** — Spring will not create or use `HttpSession` . Every request must carry its own authentication credentials (JWT). This is critical for horizontal scaling.

3. **Public auth endpoints** — `/api/auth/signup`, `/api/auth/login`, and `/api/auth/validate` must be accessible without authentication (you can't require a token to get a token).
4. **All other endpoints authenticated** — Defense in depth. Even though the Gateway handles most protection, the Auth Service protects itself.
5. **BCryptPasswordEncoder** — Adaptive hashing algorithm with salt. Cost factor 10 (default) means 2^10 = 1024 iterations. Each hash takes ~100ms, making brute force infeasible.
6. **AuthenticationManager** — Spring's built-in interface for authentication. Extracted from `AuthenticationConfiguration` for use in custom authentication logic.

3.3.2 PasswordEncoder — BCrypt Deep Dive

WHY BCrypt over MD5/SHA-256:

```
+----------------+--------------+---------------+------------------+
| Algorithm      | Speed        | Brute Force   | Production Use   |
|                | (per hash)   | Resistance    |                  |
+----------------+--------------+---------------+------------------+
| MD5            | ~1 us        | Very Weak     | NEVER            |
| SHA-256        | ~2 us        | Weak          | NEVER for pwd    |
| BCrypt (10)    | ~100 ms      | Strong        | RECOMMENDED      |
| BCrypt (12)    | ~400 ms      | Very Strong   | For high-sec     |
| Argon2         | ~500 ms      | Strongest     | Future-proof     |
+----------------+--------------+---------------+------------------+
```

How BCrypt works internally:

1. Generates a random 16-byte salt
2. Combines salt + password
3. Runs Blowfish key schedule 2^cost times (cost=10 = 1024 iterations)
4. Produces a 60-character hash: `$2a$10$N9qo8uLOickgx2ZMRZoMye...`
   - `$2a$` — BCrypt identifier
   - `10$` — Cost factor
   - Next 22 chars — Base64 encoded salt
   - Last 31 chars — Base64 encoded hash

3.3.3 AuthService — Registration and Login Logic

WHY it exists: Centralizes all authentication business logic — user creation with password hashing and login with credential verification and JWT generation.

```java
@Service
public class AuthService {
    private final UserCredentialRepository repository;
    private final PasswordEncoder passwordEncoder;
    private final JwtUtil jwtUtil;

    // SIGNUP FLOW
    public AuthResponse signup(SignupRequest request) {
        // 1. Uniqueness validation — prevent duplicate accounts
        if (repository.existsByUsername(request.getUsername())) {
            throw new RuntimeException("Username already exists");
        }
        if (repository.existsByEmail(request.getEmail())) {
```

```
            throw new RuntimeException("Email already exists");
        }

        // 2. Role determination — default USER, explicit ADMIN assignment
        UserCredential.Role role = UserCredential.Role.USER;
        if ("ADMIN".equalsIgnoreCase(request.getRole())) {
            role = UserCredential.Role.ADMIN;
        }

        // 3. Password hashing — NEVER store plaintext passwords
        UserCredential user = new UserCredential(
                request.getUsername(),
                request.getEmail(),
                passwordEncoder.encode(request.getPassword()),  // BCrypt hash
                role);
        repository.save(user);

        return new AuthResponse(null, user.getUsername(), role.name(),
                "User registered successfully");
    }

    // LOGIN FLOW
    public AuthResponse login(AuthRequest request) {
        // 1. User lookup
        UserCredential user = repository.findByUsername(request.getUsername())
                .orElseThrow(() -> new RuntimeException("User not found"));

        // 2. Password verification — BCrypt comparison
        if (!passwordEncoder.matches(request.getPassword(), user.getPassword())) {
            throw new RuntimeException("Invalid credentials");
        }

        // 3. JWT generation with username + role
        String token = jwtUtil.generateToken(user.getUsername(), user.getRole().name());

        return new AuthResponse(token, user.getUsername(), user.getRole().name(),
                "Login successful");
    }
}
```

3.3.4 UserCredential Entity — Database Schema

WHY separation from User Profile: The Auth Service stores only authentication data (credentials). Profile data (address, phone, etc.) lives in the User Service. This follows the **Database-per-Service** pattern and the **Principle of Least Privilege**.

```
@Entity
@Table(name = "user_credentials")
public class UserCredential {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```java
    @Column(unique = true, nullable = false)
    private String username;

    @Column(unique = true, nullable = false)
    private String email;

    @Column(nullable = false)
    private String password;       // BCrypt hashed — NEVER plaintext

    @Column(nullable = false)
    @Enumerated(EnumType.STRING)
    private Role role = Role.USER;

    public enum Role { USER, ADMIN }
}
```

Database Schema (auth_db):

```sql
CREATE TABLE user_credentials (
    id         BIGINT AUTO_INCREMENT PRIMARY KEY,
    username   VARCHAR(50)  NOT NULL UNIQUE,
    email      VARCHAR(100) NOT NULL UNIQUE,
    password   VARCHAR(255) NOT NULL,           -- BCrypt hash (~60 chars)
    role       ENUM('USER', 'ADMIN') NOT NULL DEFAULT 'USER',
    INDEX idx_username (username),
    INDEX idx_email (email)
);
```

Extended Schema (Production-Grade with Roles & Permissions):

```sql
-- Users table
CREATE TABLE users (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    username    VARCHAR(50)  NOT NULL UNIQUE,
    email       VARCHAR(100) NOT NULL UNIQUE,
    password    VARCHAR(255) NOT NULL,
    enabled     BOOLEAN DEFAULT TRUE,
    locked      BOOLEAN DEFAULT FALSE,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Roles table
CREATE TABLE roles (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(50) NOT NULL UNIQUE,        -- ROLE_USER, ROLE_ADMIN
    description VARCHAR(255)
);

-- Permissions table
```

```sql
CREATE TABLE permissions (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL UNIQUE,        -- user:read, order:write
    description VARCHAR(255)
);

-- Many-to-Many: Users <-> Roles
CREATE TABLE user_roles (
    user_id BIGINT NOT NULL,
    role_id BIGINT NOT NULL,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
);

-- Many-to-Many: Roles <-> Permissions
CREATE TABLE role_permissions (
    role_id       BIGINT NOT NULL,
    permission_id BIGINT NOT NULL,
    PRIMARY KEY (role_id, permission_id),
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE,
    FOREIGN KEY (permission_id) REFERENCES permissions(id) ON DELETE CASCADE
);

-- Refresh Tokens table (for token rotation)
CREATE TABLE refresh_tokens (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id     BIGINT NOT NULL,
    token       VARCHAR(500) NOT NULL UNIQUE,
    expiry_date TIMESTAMP NOT NULL,
    revoked     BOOLEAN DEFAULT FALSE,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Entity Relationship Diagram:

```
+-------------+       +--------------+       +--------------+
|   users     |       |  user_roles  |       |    roles     |
+-------------+       +--------------+       +--------------+
| id (PK)     |--+    | user_id (FK) |   +---| id (PK)      |
| username    |  +--->| role_id (FK) |<--+   | name         |
| email       |       +--------------+       | description  |
| password    |                              +------+-------+
| enabled     |       +------------------+          |
+-------------+       | role_permissions |          |
                      +------------------+          |
                      | role_id (FK)     |<---------+
                      | permission_id(FK)|------+
                      +------------------+      |
                                                v
                                        +--------------+
```

```
                                        | permissions  |
                                        +--------------+
                                        | id (PK)      |
                                        | name         |
                                        | description  |
                                        +--------------+
```

---

4. Authorization Deep Dive

4.1 What is Authorization?

Authorization answers: **"What can you access?"** After authenticating a user (knowing WHO they are), authorization determines WHAT resources they can access and WHAT operations they can perform.

4.2 RBAC (Role-Based Access Control) in This Project

WHY RBAC: It provides a manageable way to control access. Instead of assigning permissions to each user individually, permissions are grouped into roles, and roles are assigned to users.

```
Current Implementation:

USER Role:
  [YES] GET /api/products/**     (Browse products)
  [YES] GET /api/orders/**       (View own orders)
  [YES] POST /api/orders         (Place orders)
  [YES] GET /api/inventory/**    (Check stock)
  [YES] GET /api/users/profile   (View own profile)
  [NO]  /admin/**                (BLOCKED)

ADMIN Role:
  [YES] All USER permissions
  [YES] GET /admin/users         (List all users)
  [YES] PUT /admin/users/{id}/block    (Block user)
  [YES] PUT /admin/users/{id}/unblock  (Unblock user)
  [YES] DELETE /admin/users/{id}       (Delete user)
  [YES] POST /admin/products           (Create product)
  [YES] PUT /admin/products/{id}       (Update product)
  [YES] DELETE /admin/products/{id}    (Delete product)
  [YES] GET /admin/orders              (View all orders)
  [YES] PUT /admin/orders/{id}/status  (Update order status)
  [YES] POST /admin/inventory          (Add inventory)
  [YES] PUT /admin/inventory/{id}/adjust (Adjust stock)
  [YES] GET /admin/audit-logs          (View audit trail)
```

4.3 Authorization Enforcement — Three Layers

Authorization is enforced at **three distinct layers**, providing defense in depth:

```
LAYER 1: API Gateway (JwtAuthenticationFilter)
  - Extracts role from JWT
  - Blocks requests with invalid/expired tokens
  - Open endpoints bypass this filter
```

```
  - Adds X-User-Role header for downstream services

LAYER 2: Service SecurityFilterChain (URL-based)
  - .requestMatchers("/admin/**").hasRole("ADMIN")
  - .requestMatchers("/api/orders/**").permitAll()
  - Coarse-grained URL pattern matching
  - Applied by Spring Security's AuthorizationFilter

LAYER 3: Method-Level Security (@PreAuthorize)
  - @PreAuthorize("hasRole('ADMIN')")
  - Fine-grained per-method authorization
  - Applied by Spring AOP proxies
  - Can use SpEL expressions for complex logic
```

4.4 InternalAuthFilter — Gateway-to-Service Trust

WHY it exists: Business services don't have direct access to the JWT token. The API Gateway validates the token and passes user identity via HTTP headers. The `InternalAuthFilter` converts these headers into a Spring Security `Authentication` object.

WHAT problem it solves: Without this filter, Spring Security in downstream services would not know WHO the user is or WHAT role they have, even though the Gateway already validated them.

HOW it works internally:

```java
public static class InternalAuthFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
            HttpServletResponse response, FilterChain filterChain)
            throws ServletException, IOException {

        // 1. Read headers set by API Gateway's JwtAuthenticationFilter
        String username = request.getHeader("X-User-Name");
        String role = request.getHeader("X-User-Role");

        // 2. If present, create Authentication object
        if (username != null && role != null) {
            // Spring Security expects "ROLE_" prefix for hasRole() checks
            SimpleGrantedAuthority authority =
                    new SimpleGrantedAuthority("ROLE_" + role);

            // Create auth token (no credentials needed — already validated)
            UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                            username,     // principal
                            null,         // credentials (not needed)
                            Collections.singletonList(authority));  // authorities

            // 3. Set in SecurityContext
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
```

```
        // 4. Continue filter chain (with or without auth)
        filterChain.doFilter(request, response);
    }
}
```

Critical Detail — ROLE_ Prefix: Spring Security's `hasRole("ADMIN")` internally checks for `ROLE_ADMIN` authority. The filter must prepend `ROLE_` to match. Without this, `hasRole()` checks always fail.

4.5 Method-Level Security with @PreAuthorize

WHY used in addition to URL-based rules: URL patterns are coarse-grained. `@PreAuthorize` provides fine-grained, method-level control with SpEL expressions.

```
@RestController
@RequestMapping("/admin/users")
public class AdminUserController {

    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")       // Only ADMIN can list all users
    public ResponseEntity<List<UserProfileDto>> getAllUsers() {
        return ResponseEntity.ok(userService.getAllProfiles());
    }

    @PutMapping("/{id}/block")
    @PreAuthorize("hasRole('ADMIN')")       // Only ADMIN can block users
    public ResponseEntity<Void> blockUser(@PathVariable Long id,
            @RequestHeader("X-User-Name") String adminUsername) {
        userService.blockUser(id);
        // Audit logging — WHO did WHAT to WHOM
        publishAudit(adminUsername, "BLOCK", id.toString(), "Blocked user");
        return ResponseEntity.noContent().build();
    }
}
```

Requires `@EnableMethodSecurity` on the SecurityConfig class:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity    // Activates @PreAuthorize, @PostAuthorize, @Secured
public class SecurityConfig { ... }
```

4.6 Inventory Service — Fine-Grained RBAC Example

The Inventory Service demonstrates the most granular RBAC configuration:

```
.authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.OPTIONS).permitAll()
        .requestMatchers(HttpMethod.GET, "/api/inventory/**").permitAll()
        .requestMatchers(HttpMethod.POST, "/api/inventory").hasRole("ADMIN")
        .requestMatchers(HttpMethod.PUT, "/api/inventory/**").hasRole("ADMIN")
```

```
        .requestMatchers("/admin/inventory/**").hasRole("ADMIN")
        .anyRequest().authenticated());
```

Access Control Matrix:

```
+-------------------------------------+----------+-------+
| Endpoint                            | USER     | ADMIN |
+-------------------------------------+----------+-------+
| GET  /api/inventory/{productId}     | YES      | YES   |
| GET  /api/inventory/check/{id}      | YES      | YES   |
| POST /api/inventory                 | NO       | YES   |
| PUT  /api/inventory/{id}/adjust     | NO       | YES   |
| GET  /admin/inventory               | NO       | YES   |
| POST /admin/inventory               | NO       | YES   |
+-------------------------------------+----------+-------+
```

---

5. JWT Deep Dive

5.1 JWT Internal Structure

A JWT (JSON Web Token) consists of three Base64URL-encoded parts separated by dots:

```
eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiQURNSU4iLCJzdWIiOiJhZG1pbjEi...

Structure:
+-------------------------------------------------+
|                    HEADER                       |
|  {                                              |
|    "alg": "HS256",        <- Signing algorithm  |
|    "typ": "JWT"           <- Token type         |
|  }                                              |
+-------------------------------------------------+
|                    PAYLOAD (Claims)             |
|  {                                              |
|    "sub": "admin1",       <- Subject (username) |
|    "role": "ADMIN",       <- Custom claim (role)|
|    "iat": 1709500000,     <- Issued At timestamp|
|    "exp": 1709586400      <- Expiration timestamp|
|  }                                              |
+-------------------------------------------------+
|                    SIGNATURE                    |
|  HMACSHA256(                                    |
|    base64UrlEncode(header) + "." +              |
|    base64UrlEncode(payload),                    |
|    secret_key                                   |
|  )                                              |
|                                                 |
|  Purpose: Ensures the token hasn't been         |
|  tampered with. If ANY character in the header  |
|  or payload changes, the signature won't match. |
+-------------------------------------------------+
```

5.2 Token Lifecycle

```
PHASE 1: TOKEN CREATION (Auth Service)
  User logs in -> AuthService.login() -> JwtUtil.generateToken()
  - Claims set: sub=username, role=USER/ADMIN
  - iat = current time
  - exp = current time + 86400000ms (24 hours)
  - Signed with HMAC-SHA256 using shared secret
  - Token returned to client

PHASE 2: TOKEN USAGE (Every API Call)
  Client includes: Authorization: Bearer <token>
  - Gateway JwtAuthenticationFilter intercepts
  - Extracts token from header (removes "Bearer " prefix)
  - Validates: signature + expiration
  - Extracts: username + role from claims
  - Adds: X-User-Name, X-User-Role headers
  - Forwards to downstream service

PHASE 3: TOKEN EXPIRATION
  After 24 hours (configurable via jwt.expiration):
  - JwtUtil.isTokenExpired() returns true
  - Gateway returns 401 Unauthorized
  - Client must re-authenticate (login again)
  - Currently: No refresh token mechanism
  - Production: Should implement refresh tokens

PHASE 4: TOKEN INVALIDATION (Production Enhancement)
  Stateless JWT limitation: Cannot invalidate before expiry
  Production solutions:
  - Redis-based token blacklist
  - Short-lived access tokens + refresh tokens
  - Token version field in user record
```

5.3 JwtUtil — Auth Service Implementation

```java
@Component
public class JwtUtil {
    private final SecretKey secretKey;
    private final long expiration;

    public JwtUtil(@Value("${jwt.secret}") String secret,
            @Value("${jwt.expiration}") long expiration) {
        // Decode Base64-encoded secret from application.yml
        byte[] keyBytes = Base64.getDecoder().decode(secret);
        // Create HMAC-SHA256 key
        this.secretKey = Keys.hmacShaKeyFor(keyBytes);
        this.expiration = expiration;   // 86400000ms = 24 hours
    }
```

```java
    public String generateToken(String username, String role) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);   // Custom claim for authorization

        return Jwts.builder()
                .claims(claims)                             // Set custom claims
                .subject(username)                          // Standard "sub" claim
                .issuedAt(new Date())                       // Standard "iat" claim
                .expiration(new Date(
                        System.currentTimeMillis() + expiration))  // "exp" claim
                .signWith(secretKey)                        // HMAC-SHA256 signature
                .compact();                                 // Serialize to string
    }

    public Claims extractAllClaims(String token) {
        return Jwts.parser()
                .verifyWith(secretKey)     // Verify signature with same key
                .build()
                .parseSignedClaims(token)  // Parse and validate
                .getPayload();             // Return claims
    }

    public boolean isTokenValid(String token) {
        try {
            return !isTokenExpired(token);
        } catch (Exception e) {
            return false;   // Any parsing error = invalid token
        }
    }
}
```

5.4 JwtUtil — Gateway Implementation (Validation Only)

WHY a separate JwtUtil in the Gateway: The Gateway only validates tokens (never generates them). Keeping generation logic in the Auth Service follows the Single Responsibility Principle.

```java
@Component
public class JwtUtil {
    private final SecretKey secretKey;

    public JwtUtil(@Value("${jwt.secret}") String secret) {
        byte[] keyBytes = Base64.getDecoder().decode(secret);
        this.secretKey = Keys.hmacShaKeyFor(keyBytes);
        // NOTE: Same secret key as Auth Service — critical for validation
    }

    // Only validation methods — no generateToken()

    public String extractUsername(String token) {
        return extractAllClaims(token).getSubject();
    }
```

```
    public String extractRole(String token) {
        return extractAllClaims(token).get("role", String.class);
    }

    public boolean isTokenValid(String token) {
        try { return !isTokenExpired(token); }
        catch (Exception e) { return false; }
    }
}
```

5.5 Symmetric vs Asymmetric JWT Signing

```
SYMMETRIC (HMAC-SHA256) — Current Implementation:
  - Same secret key for signing AND verification
  - Simpler setup — one key to manage
  - Both Auth Service and Gateway must have the same key
  - Risk: If key leaks from ANY service, attacker can forge tokens
  - Best for: Internal microservices where all services are trusted

  Auth Service --[signs with SECRET]--> JWT
  Gateway      --[verifies with SECRET]--> Valid/Invalid

  jwt.secret: YTJiM2M0ZDVlNmY3ZzhoOWkwajFrMmwzbTRuNW82cDdxOHI5czB0...
  (Same in both application.yml files)

ASYMMETRIC (RSA/ECDSA) — Recommended for Production:
  - Private key for signing (Auth Service only)
  - Public key for verification (Gateway, any service)
  - Stronger security — private key never leaves Auth Service
  - Public key can be distributed freely
  - Best for: Third-party integrations, zero-trust architectures

  Auth Service --[signs with PRIVATE KEY]--> JWT
  Gateway      --[verifies with PUBLIC KEY]--> Valid/Invalid
  Other Service--[verifies with PUBLIC KEY]--> Valid/Invalid
```

---

6. Step-by-Step Implementation Guide

Step 1 — Database Design

See Section 3.3.4 for complete schema design including:

- `user_credentials` table (current implementation)
- Extended `users`, `roles`, `permissions`, `user_roles`, `role_permissions` tables
- `refresh_tokens` table for token rotation

Step 2 — Auth Service Setup

Required Dependencies (pom.xml):

```xml
<dependencies>
    <!-- Spring Boot Starter Web -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <!-- Spring Security -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <!-- Spring Data JPA -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <!-- MySQL Connector -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <!-- JJWT (JWT library) -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
    <!-- Bean Validation -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <!-- Eureka Client -->
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
```

```
    </dependency>
</dependencies>
```

Application Configuration (application.yml):

```yaml
server:
  port: 8081

spring:
  application:
    name: auth-service
  datasource:
    url: jdbc:mysql://localhost:3306/auth_db
    username: orderuser
    password: orderpass
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/

jwt:
  secret: YTJiM2M0ZDVlNmY3ZzhoOWkwajFrMmwzbTRuNW82cDdxOHI5czB0MXUydjN3NHg1eTZ6
  expiration: 86400000    # 24 hours
```

Component Interaction Map:

```
AuthController --> AuthService --> UserCredentialRepository
    |                  |                        |
    |                  +---> PasswordEncoder     |
    |                  |    (BCrypt hashing)     |
    |                  |                        v
    |                  +--> JwtUtil             auth_db
    |                       (Token generation)  (MySQL)
    v
GlobalExceptionHandler
    (Consistent error responses)
```

Step 3 — API Gateway Security

Gateway Security Components:

```
+-------------------------------------------------------+
|                    API Gateway Filter Pipeline            |
|                                                          |
|  Request --> CorsWebFilter                               |
```

```
|             |  Handles OPTIONS preflight                  |
|             |  Allows localhost:3000 origin               |
|             v                                             |
|         JwtAuthenticationFilter (per-route)               |
|             |  Checks OPEN_ENDPOINTS list                 |
|             |  Extracts Authorization header              |
|             |  Validates Bearer token format              |
|             |  Calls JwtUtil.isTokenValid()               |
|             |  Extracts username and role                 |
|             |  Mutates request: adds X-User-Name/Role     |
|             v                                             |
|         CircuitBreaker                                    |
|             |  Prevents cascade failures                  |
|             |  Falls back to /fallback/* on failure       |
|             v                                             |
|         Route to Downstream Service                       |
+-----------------------------------------------------------+
```

JwtAuthenticationFilter (Complete Implementation):

```java
@Component
public class JwtAuthenticationFilter
        extends AbstractGatewayFilterFactory<JwtAuthenticationFilter.Config> {

    private final JwtUtil jwtUtil;

    private static final List<String> OPEN_ENDPOINTS = List.of(
            "/api/auth/signup",
            "/api/auth/login",
            "/api/auth/validate",
            "/fallback");

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            ServerHttpRequest request = exchange.getRequest();
            String path = request.getURI().getPath();

            // 1. Skip auth for public endpoints
            if (isOpenEndpoint(path)) return chain.filter(exchange);

            // 2. Skip JWT for CORS preflight
            if ("OPTIONS".equalsIgnoreCase(request.getMethod().name()))
                return chain.filter(exchange);

            // 3. Validate Authorization header exists
            if (!request.getHeaders().containsKey(HttpHeaders.AUTHORIZATION))
                return onError(exchange, "Missing Authorization header",
                        HttpStatus.UNAUTHORIZED);

            // 4. Validate Bearer format
            String authHeader = request.getHeaders()
```

```java
                .getFirst(HttpHeaders.AUTHORIZATION);
            if (authHeader == null || !authHeader.startsWith("Bearer "))
                return onError(exchange, "Invalid Authorization header",
                        HttpStatus.UNAUTHORIZED);

            // 5. Extract and validate JWT
            String token = authHeader.substring(7);
            try {
                if (!jwtUtil.isTokenValid(token))
                    return onError(exchange, "Token expired or invalid",
                            HttpStatus.UNAUTHORIZED);

                // 6. Extract claims and propagate via headers
                String username = jwtUtil.extractUsername(token);
                String role = jwtUtil.extractRole(token);

                ServerHttpRequest modifiedRequest = request.mutate()
                        .header("X-User-Name", username)
                        .header("X-User-Role", role)
                        .build();

                return chain.filter(exchange.mutate()
                        .request(modifiedRequest).build());
            } catch (Exception e) {
                return onError(exchange, "Token validation failed",
                        HttpStatus.UNAUTHORIZED);
            }
        };
    }
}
```

Route Configuration with JWT Filter (application.yml):

```yaml
spring:
  cloud:
    gateway:
      routes:
        # Auth Service — NO JWT filter (public endpoints)
        - id: auth-service
          uri: lb://auth-service
          predicates:
            - Path=/api/auth/**
          filters:
            - name: CircuitBreaker
              args:
                name: authServiceCircuitBreaker
                fallbackUri: forward:/fallback/auth

        # All other services — JWT filter applied
        - id: order-service
          uri: lb://order-service
```

```yaml
        predicates:
          - Path=/api/orders/**, /admin/orders/**
        filters:
          - JwtAuthenticationFilter           # JWT validation
          - name: CircuitBreaker
            args:
              name: orderServiceCircuitBreaker
              fallbackUri: forward:/fallback/generic
```

Step 4 — Securing Business Services

Each business service follows this consistent pattern:

```java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity          // Enables @PreAuthorize
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
                    session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(new InternalAuthFilter(),
                    UsernamePasswordAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth
                    .requestMatchers(HttpMethod.OPTIONS).permitAll()
                    .requestMatchers("/api/<service>/**").permitAll()
                    .requestMatchers("/admin/**").hasRole("ADMIN")
                    .anyRequest().authenticated());
        return http.build();
    }

    // InternalAuthFilter converts Gateway headers to Authentication
    public static class InternalAuthFilter extends OncePerRequestFilter {
        // ... (See Section 4.4 for full implementation)
    }
}
```

Step 5 — Inter-Service Communication Security

Current implementation: Services communicate via Kafka events (asynchronous) and direct HTTP calls.

```
INTER-SERVICE COMMUNICATION SECURITY:

1. KAFKA-BASED (Event-Driven):
   Order Service -> [Kafka Topic] -> Inventory Service
   - No JWT needed — messages are internal
   - Security: Kafka ACLs in production
```

```
   - Trust boundary: Kafka cluster is internal

2. DIRECT HTTP (Synchronous):
   Order Service -> Inventory Service (stock check)
   - JWT should be forwarded in Authorization header
   - Gateway headers (X-User-Name, X-User-Role) can be passed

3. RECOMMENDED: Service-to-Service Token
   - Create a special "SYSTEM" role JWT
   - Services authenticate to each other using this token
   - Short expiry (5 minutes) for internal calls
```

Forwarding JWT in Inter-Service Calls (Production Pattern):

```java
@Component
public class JwtRequestInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        ServletRequestAttributes attrs = (ServletRequestAttributes)
                RequestContextHolder.getRequestAttributes();
        if (attrs != null) {
            String authHeader = attrs.getRequest()
                    .getHeader(HttpHeaders.AUTHORIZATION);
            if (authHeader != null) {
                template.header(HttpHeaders.AUTHORIZATION, authHeader);
            }
        }
    }
}
```

---

7. Code Walkthrough

7.1 Complete Request Lifecycle — End to End

Let's trace a complete request from login to accessing a protected admin endpoint:

```
STEP 1: User Registration
=========================
POST http://localhost:8080/api/auth/signup
Content-Type: application/json

{
    "username": "admin1",
    "email": "admin1@example.com",
    "password": "securePass123",
    "role": "ADMIN"
}

INTERNAL FLOW:
  Client -> Gateway (port 8080)
    -> Route: /api/auth/** -> NO JwtAuthenticationFilter
```

```
    -> Forward to: lb://auth-service (port 8081)
      -> AuthController.signup()
        -> AuthService.signup()
          -> Check: existsByUsername("admin1") = false
          -> Check: existsByEmail("admin1@example.com") = false
          -> Role: ADMIN (explicit assignment)
          -> Hash: BCrypt.encode("securePass123")
             = "$2a$10$N9qo8uLOickgx2ZMRZoMye..."
          -> Save to user_credentials table
        -> Return: {token: null, username: "admin1",
                    role: "ADMIN", message: "User registered successfully"}

STEP 2: User Login
===================
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
    "username": "admin1",
    "password": "securePass123"
}

INTERNAL FLOW:
  Client -> Gateway -> Auth Service
    -> AuthController.login()
      -> AuthService.login()
        -> Find user by username: SELECT * FROM user_credentials
                                  WHERE username='admin1'
        -> Verify: BCrypt.matches("securePass123", storedHash) = true
        -> Generate JWT:
           Header:  {"alg":"HS256","typ":"JWT"}
           Payload: {"sub":"admin1","role":"ADMIN",
                     "iat":1709500000,"exp":1709586400}
           Sign with: HMAC-SHA256(header.payload, secretKey)
      -> Return: {token: "eyJhbGciOi...", username: "admin1",
                  role: "ADMIN", message: "Login successful"}

STEP 3: Access Protected Admin Endpoint
========================================
GET http://localhost:8080/admin/orders
Authorization: Bearer eyJhbGciOi...

INTERNAL FLOW:
  Client -> Gateway (port 8080)
    -> CorsWebFilter: Pass (not a preflight request)
    -> Route match: /admin/orders/** -> order-service route
    -> JwtAuthenticationFilter:
       a. Not an OPEN_ENDPOINT: proceed with validation
       b. Not OPTIONS: proceed
       c. Authorization header present: YES
       d. Starts with "Bearer ": YES
       e. Extract token: "eyJhbGciOi..."
```

```
      f. JwtUtil.isTokenValid(token):
         - Parse with HMAC-SHA256 key
         - Verify signature: MATCH
         - Check expiration: NOT EXPIRED
         - Result: VALID
      g. Extract claims:
         - username = "admin1"
         - role = "ADMIN"
      h. Mutate request:
         - Add header: X-User-Name=admin1
         - Add header: X-User-Role=ADMIN
   -> Forward to: lb://order-service (port 8084)

  Order Service receives request:
   -> SecurityFilterChain:
      -> InternalAuthFilter (OncePerRequestFilter):
         - Read X-User-Name: "admin1"
         - Read X-User-Role: "ADMIN"
         - Create: ROLE_ADMIN authority
         - Set SecurityContext authentication
      -> AuthorizationFilter:
         - Path: /admin/orders
         - Rule: /admin/** requires hasRole("ADMIN")
         - User has: ROLE_ADMIN
         - Result: ACCESS GRANTED
   -> AdminOrderController:
      -> @PreAuthorize("hasRole('ADMIN')"):
         - Check SecurityContext
         - User has ROLE_ADMIN
         - Result: ALLOWED
      -> Execute business logic
      -> Return response
```

7.2 Error Scenarios

```
SCENARIO 1: Missing Authorization Header
  GET /api/orders/123 (no Authorization header)
  -> Gateway JwtAuthenticationFilter
  -> headers.containsKey(AUTHORIZATION) = false
  -> Return 401: {"error": "Missing Authorization header"}

SCENARIO 2: Expired Token
  GET /api/orders/123
  Authorization: Bearer <expired_token>
  -> Gateway JwtAuthenticationFilter
  -> JwtUtil.isTokenValid(token)
     -> extractAllClaims(token).getExpiration().before(new Date())
     -> true (expired)
  -> Return 401: {"error": "Token expired or invalid"}

SCENARIO 3: Tampered Token
```

```
  GET /api/orders/123
  Authorization: Bearer <modified_payload_token>
  -> Gateway JwtAuthenticationFilter
  -> JwtUtil.isTokenValid(token)
     -> Jwts.parser().verifyWith(secretKey).build().parseSignedClaims(token)
     -> THROWS SignatureException (payload modified, signature mismatch)
  -> Caught by catch(Exception e)
  -> Return 401: {"error": "Token validation failed"}

SCENARIO 4: USER Accessing ADMIN Endpoint
  GET /admin/orders (with USER role token)
  -> Gateway: JWT valid, adds X-User-Role=USER
  -> Order Service: InternalAuthFilter sets ROLE_USER
  -> SecurityFilterChain: /admin/** requires hasRole("ADMIN")
  -> ROLE_USER does NOT match
  -> Return 403 Forbidden

SCENARIO 5: Invalid Password
  POST /api/auth/login {username: "admin1", password: "wrong"}
  -> Auth Service -> AuthService.login()
  -> BCrypt.matches("wrong", storedHash) = false
  -> throw RuntimeException("Invalid credentials")
  -> GlobalExceptionHandler -> 400 Bad Request:
     {"message": "Invalid credentials", "status": 400, "timestamp": "..."}
```

7.3 CORS Configuration Walkthrough

WHY CORS configuration is critical: The React frontend (localhost:3000) makes API calls to the Gateway (localhost:8080). Browsers enforce Same-Origin Policy, blocking cross-origin requests unless the server explicitly allows them.

```java
@Configuration
public class CorsConfig {
    @Bean
    public CorsWebFilter corsWebFilter() {
        CorsConfiguration corsConfig = new CorsConfiguration();
        // Exact origin required for allowCredentials(true)
        corsConfig.setAllowedOrigins(
                Collections.singletonList("http://localhost:3000"));
        corsConfig.setMaxAge(3600L);  // Cache preflight for 1 hour
        corsConfig.setAllowedMethods(Arrays.asList(
                "GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));
        corsConfig.addAllowedHeader("*");      // Allow all headers
        corsConfig.setAllowCredentials(true);   // Allow cookies/auth headers

        UrlBasedCorsConfigurationSource source =
                new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", corsConfig);
        return new CorsWebFilter(source);
    }
}
```

---

8. Production-Grade Enhancements

8.1 Refresh Token Mechanism

WHY: Short-lived access tokens (15 min) limit the damage window if stolen, but users shouldn't re-login every 15 minutes.

```
ACCESS TOKEN + REFRESH TOKEN FLOW:
==================================

1. Login Response:
   {
     "accessToken": "eyJhbG...",      // 15 min expiry
     "refreshToken": "dGhpcyBpcy...", // 7 day expiry, stored in DB
     "expiresIn": 900
   }

2. Normal API Call (with access token):
   Authorization: Bearer <accessToken>

3. Access Token Expires (after 15 min):
   -> Gateway returns 401

4. Client Sends Refresh Request:
   POST /api/auth/refresh
   { "refreshToken": "dGhpcyBpcy..." }

5. Auth Service:
   -> Find refresh token in DB
   -> Check if revoked/expired
   -> Generate new access token + new refresh token
   -> Revoke old refresh token (token rotation)
   -> Return new tokens

6. Client continues with new access token
```

Implementation:

```java
@Entity
@Table(name = "refresh_tokens")
public class RefreshToken {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long userId;
    private String token;       // UUID or cryptographic random
    private Instant expiryDate;
    private boolean revoked = false;
}

@Service
public class RefreshTokenService {
```

```java
    public RefreshToken createRefreshToken(Long userId) {
        RefreshToken token = new RefreshToken();
        token.setUserId(userId);
        token.setToken(UUID.randomUUID().toString());
        token.setExpiryDate(Instant.now().plusMillis(604800000)); // 7 days
        return repository.save(token);
    }

    public AuthResponse refreshAccessToken(String refreshTokenStr) {
        RefreshToken refreshToken = repository.findByToken(refreshTokenStr)
                .orElseThrow(() -> new RuntimeException("Invalid refresh token"));

        if (refreshToken.isRevoked() ||
                refreshToken.getExpiryDate().isBefore(Instant.now())) {
            throw new RuntimeException("Refresh token expired or revoked");
        }

        // Revoke old refresh token (rotation)
        refreshToken.setRevoked(true);
        repository.save(refreshToken);

        // Generate new tokens
        UserCredential user = userRepository.findById(refreshToken.getUserId())
                .orElseThrow();
        String newAccessToken = jwtUtil.generateToken(
                user.getUsername(), user.getRole().name());
        RefreshToken newRefreshToken = createRefreshToken(user.getId());

        return new AuthResponse(newAccessToken, user.getUsername(),
                user.getRole().name(), newRefreshToken.getToken());
    }
}
```

8.2 Token Blacklisting

WHY: JWTs are stateless — once issued, they're valid until expiration. If a user logs out or their account is compromised, we need a way to invalidate tokens before they expire.

```
REDIS-BASED TOKEN BLACKLIST:
============================

1. User logs out:
   POST /api/auth/logout
   Authorization: Bearer <token>

2. Auth Service:
   -> Extract token's JTI (JWT ID) and remaining TTL
   -> Store in Redis: SET blacklist:<jti> true EX <remaining_ttl>

3. Gateway checks blacklist on every request:
   -> After JWT validation passes
   -> Check Redis: EXISTS blacklist:<jti>
```

```
   -> If exists: return 401 (token is blacklisted)
   -> If not: proceed normally

4. Redis auto-cleans: TTL ensures entries expire when token would have
```

```java
@Service
public class TokenBlacklistService {
    private final StringRedisTemplate redisTemplate;

    public void blacklist(String token) {
        Claims claims = jwtUtil.extractAllClaims(token);
        String jti = claims.getId();
        long ttl = claims.getExpiration().getTime() - System.currentTimeMillis();
        if (ttl > 0) {
            redisTemplate.opsForValue()
                    .set("blacklist:" + jti, "true", ttl, TimeUnit.MILLISECONDS);
        }
    }

    public boolean isBlacklisted(String token) {
        Claims claims = jwtUtil.extractAllClaims(token);
        return Boolean.TRUE.equals(
                redisTemplate.hasKey("blacklist:" + claims.getId()));
    }
}
```

8.3 Rate Limiting

WHY: Prevents brute force attacks on login endpoints and DDoS attacks on the API.

```yaml
# Gateway application.yml — Resilience4j Rate Limiter
resilience4j:
  ratelimiter:
    configs:
      default:
        limit-for-period: 100          # 100 requests
        limit-refresh-period: 1m       # per minute
        timeout-duration: 0s           # Reject immediately if limit exceeded
    instances:
      loginRateLimiter:
        limit-for-period: 5            # 5 login attempts
        limit-refresh-period: 1m       # per minute per IP
```

8.4 Brute Force Protection

```java
@Service
public class LoginAttemptService {
    private final Map<String, Integer> attemptsCache = new ConcurrentHashMap<>();
    private static final int MAX_ATTEMPTS = 5;
    private static final long LOCK_TIME_MS = 900000; // 15 minutes
```

```java
    public void loginFailed(String username) {
        int attempts = attemptsCache.getOrDefault(username, 0);
        attemptsCache.put(username, attempts + 1);
    }

    public boolean isBlocked(String username) {
        return attemptsCache.getOrDefault(username, 0) >= MAX_ATTEMPTS;
    }

    public void loginSucceeded(String username) {
        attemptsCache.remove(username);
    }
}
```

8.5 Password Policies

```java
public class PasswordValidator {
    public static void validate(String password) {
        if (password.length() < 8)
            throw new ValidationException("Min 8 characters");
        if (!password.matches(".*[A-Z].*"))
            throw new ValidationException("Must contain uppercase letter");
        if (!password.matches(".*[a-z].*"))
            throw new ValidationException("Must contain lowercase letter");
        if (!password.matches(".*[0-9].*"))
            throw new ValidationException("Must contain digit");
        if (!password.matches(".*[!@#$%^&*].*"))
            throw new ValidationException("Must contain special character");
    }
}
```

8.6 Audit Logging

Already implemented in this project via Kafka-based audit events:

```java
private void publishAudit(String actor, String action,
        String resourceId, String details) {
    AuditLogEvent event = new AuditLogEvent(
            UUID.randomUUID().toString(),
            actor,          // WHO (from X-User-Name header)
            action,         // WHAT (BLOCK, UNBLOCK, DELETE)
            "USER",         // Resource type
            resourceId,     // WHICH resource
            details,        // Human-readable description
            LocalDateTime.now());  // WHEN
    eventProducer.publishAuditLog(event);
}
```

8.7 Secure Headers Configuration

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {
    http
        .headers(headers -> headers
            .contentSecurityPolicy(csp ->
                    csp.policyDirectives("default-src 'self'"))
            .frameOptions(frame -> frame.deny())
            .xssProtection(xss -> xss.headerValue(
                    XXssProtectionHeaderWriter.HeaderValue.ENABLED))
            .contentTypeOptions(Customizer.withDefaults())  // nosniff
            .httpStrictTransportSecurity(hsts -> hsts
                    .includeSubDomains(true)
                    .maxAgeInSeconds(31536000)));
    // ... rest of config
}
```

8.8 CSRF Explanation in Stateless Systems

WHY CSRF is disabled in this architecture:

```
CSRF Attack Mechanism:
  1. User logs into your-app.com (session cookie stored)
  2. User visits evil-site.com
  3. evil-site.com has: <form action="your-app.com/transfer" method="POST">
  4. Browser automatically sends session cookie
  5. Server thinks it's a legitimate request

WHY JWT IS IMMUNE:
  1. JWT is stored in localStorage (not cookies)
  2. JWT must be explicitly added to Authorization header
  3. JavaScript from evil-site.com CANNOT access your-app.com's localStorage
     (Same-Origin Policy)
  4. Therefore, CSRF attacks are impossible with Bearer token authentication
  5. => Safe to disable CSRF: .csrf(csrf -> csrf.disable())

EXCEPTION: If you store JWT in cookies:
  - You MUST enable CSRF protection
  - Or use SameSite=Strict cookie attribute
```

8.9 Key Rotation Strategy for JWT

```
KEY ROTATION PROCESS:
=====================

Phase 1: Generate New Key
  - Create new HMAC-SHA256 key (Key-B)
  - Keep old key (Key-A) active
```

```
Phase 2: Dual-Key Validation Period
  - Sign new tokens with Key-B
  - Validate tokens with BOTH Key-A and Key-B
  - Duration: >= max token lifetime (24 hours)

Phase 3: Retire Old Key
  - All Key-A tokens have expired
  - Remove Key-A from validation list
  - Only Key-B remains active

Implementation:
  - Store keys in HashiCorp Vault or AWS Secrets Manager
  - Use key versioning (kid claim in JWT header)
  - Automate rotation with scheduled jobs
```

---

9. Security Best Practices

9.1 Production Checklist

```
[CRITICAL] Use HTTPS everywhere (TLS 1.3)
[CRITICAL] Never commit JWT secret keys to version control
[CRITICAL] Use environment variables or Vault for secrets
[CRITICAL] Hash passwords with BCrypt (cost >= 10)
[CRITICAL] Validate all input (bean validation + custom)

[HIGH] Implement refresh token rotation
[HIGH] Set short access token expiry (15-30 min)
[HIGH] Implement token blacklisting for logout
[HIGH] Add rate limiting on auth endpoints
[HIGH] Enable audit logging for all security events
[HIGH] Use asymmetric keys (RSA) in production

[MEDIUM] Implement account lockout after failed attempts
[MEDIUM] Add password complexity requirements
[MEDIUM] Set secure HTTP headers (CSP, HSTS, X-Frame-Options)
[MEDIUM] Implement IP-based blocking for suspicious activity
[MEDIUM] Regular security dependency updates

[LOW] Add multi-factor authentication (MFA)
[LOW] Implement session management UI
[LOW] Add geographic login anomaly detection
```

9.2 Secret Management

```
NEVER DO THIS:
  jwt:
    secret: hardcoded_value_in_yml   (COMMITTED TO GIT!)

DO THIS INSTEAD:
```

```
  jwt:
    secret: ${JWT_SECRET}           (Environment variable)

PRODUCTION:
  - Use HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault
  - Rotate secrets periodically
  - Audit secret access
  - Different secrets per environment (dev/staging/prod)
```

---

10. Common Mistakes & Anti-Patterns

10.1 Security Anti-Patterns

```
MISTAKE 1: Storing JWT in cookies without SameSite
  WRONG:  response.addCookie(new Cookie("token", jwt));
  RIGHT:  Use Authorization header OR set SameSite=Strict, HttpOnly, Secure

MISTAKE 2: Not validating token on every request
  WRONG:  Validate only on login, trust subsequent requests
  RIGHT:  Gateway validates JWT on EVERY request to protected endpoints

MISTAKE 3: Using the same secret for all environments
  WRONG:  Same jwt.secret in dev, staging, and production
  RIGHT:  Different secrets per environment, stored in Vault

MISTAKE 4: Not handling token expiration gracefully
  WRONG:  Show generic error when token expires
  RIGHT:  Return 401 with clear message, implement refresh mechanism

MISTAKE 5: Putting sensitive data in JWT payload
  WRONG:  claims.put("password", user.getPassword());
  RIGHT:  Only include: sub (username), role, iat, exp

MISTAKE 6: Missing ROLE_ prefix in InternalAuthFilter
  WRONG:  new SimpleGrantedAuthority(role);     // "ADMIN"
  RIGHT:  new SimpleGrantedAuthority("ROLE_" + role);  // "ROLE_ADMIN"
  WHY:    hasRole("ADMIN") checks for "ROLE_ADMIN" internally

MISTAKE 7: Not securing actuator endpoints
  WRONG:  management.endpoints.web.exposure.include: *
  RIGHT:  Expose only health,info and secure with role-based access

MISTAKE 8: Trusting X-User-Name/X-User-Role headers from external clients
  WRONG:  Anyone can set these headers when calling services directly
  RIGHT:  Services should ONLY be accessible through the Gateway
          (use network policies/firewalls to enforce this)

MISTAKE 9: Logging JWT tokens
  WRONG:  log.info("Token: {}", token);
  RIGHT:  log.info("Token validated for user: {}", username);
```

```
MISTAKE 10: Not validating request body
  WRONG:  public AuthResponse signup(@RequestBody SignupRequest request)
  RIGHT:  public AuthResponse signup(@Valid @RequestBody SignupRequest request)
```

---

11. Interview Preparation Questions

11.1 Authentication & Authorization Questions

Q1: Explain the difference between Authentication and Authorization.

Authentication verifies WHO you are (identity verification through credentials). Authorization determines WHAT you can access (permission verification through roles/permissions). In our system:

- Authentication happens at the Auth Service (login with username/password, JWT generation)
- Authorization happens at three layers: Gateway (token validation), Service SecurityConfig (URL patterns), and Controller (@PreAuthorize annotations)

Q2: Why did you choose JWT over session-based authentication?

In a microservices architecture with 8+ services, session-based auth requires a shared session store (Redis/DB) that all services must access on every request. JWT is self-contained — it carries user identity and can be validated cryptographically without any database lookup. This enables horizontal scaling, eliminates a shared state dependency, and reduces latency.

Q3: How does your API Gateway enforce security?

The Gateway uses a custom `JwtAuthenticationFilter` (extending `AbstractGatewayFilterFactory`) that:

1. Whitelists public endpoints (auth login/signup)
2. Extracts the Bearer token from the Authorization header
3. Validates the JWT signature using HMAC-SHA256
4. Checks token expiration
5. Extracts username and role from claims
6. Adds `X-User-Name` and `X-User-Role` headers to the request
7. Forwards the mutated request to the downstream service

If any validation fails, it returns 401 Unauthorized as a JSON error response.

Q4: How do downstream services know who the user is?

Each business service has an `InternalAuthFilter` (extends `OncePerRequestFilter`) that:

1. Reads `X-User-Name` and `X-User-Role` headers set by the Gateway
2. Creates a `UsernamePasswordAuthenticationToken` with the role prefixed as `ROLE_<role>`
3. Sets it in the `SecurityContextHolder`
4. This makes the user's identity available to `@PreAuthorize` and `SecurityContext`

Q5: Why is CSRF disabled? Is it safe?

CSRF attacks exploit session cookies that browsers automatically send. Since we use Bearer tokens (not cookies), the JWT must be explicitly added to the Authorization header. JavaScript from a malicious site cannot access our app's localStorage (Same-Origin Policy), so CSRF attacks are impossible. Therefore, disabling CSRF is safe.

Q6: What happens when a JWT token expires?

1. Client sends a request with an expired token

2. Gateway's JwtUtil.isTokenExpired() returns true
3. JwtAuthenticationFilter returns 401 Unauthorized
4. Client receives the 401 response
5. Client redirects to login page In production, a refresh token mechanism should be implemented so users don't need to re-enter credentials.

Q7: How do you prevent a USER from accessing ADMIN endpoints?

Three-layer defense:

1. **Gateway**: Forwards the user's role (e.g., "USER") in X-User-Role header
2. **SecurityFilterChain**: `.requestMatchers("/admin/**").hasRole("ADMIN")` blocks at URL level
3. **@PreAuthorize**: `@PreAuthorize("hasRole('ADMIN')")` blocks at method level Spring Security checks the `ROLE_USER` authority against the required `ROLE_ADMIN` and returns 403 Forbidden.

Q8: Why BCrypt and not SHA-256 for password hashing?

SHA-256 is fast (~~2 microseconds per hash) — an attacker with a GPU can try billions of passwords per second. BCrypt is deliberately slow (~~100ms with cost=10) because it runs 1024 rounds of Blowfish key scheduling. It also automatically generates and stores a unique salt per password, preventing rainbow table attacks.

Q9: What is the ROLE_ prefix issue and how did you solve it?

Spring Security's `hasRole("ADMIN")` internally checks for the authority `ROLE_ADMIN`. When the InternalAuthFilter creates the authority from the header value "ADMIN", it must prepend "ROLE_" to create `new SimpleGrantedAuthority("ROLE_" + role)`. Without this prefix, `hasRole()` never matches and all role-based access is denied.

Q10: How would you implement logout in a JWT-based system?

Since JWT is stateless, we need a token blacklist:

1. User sends logout request with their token
2. Extract the JTI (JWT ID) and remaining TTL from the token
3. Store the JTI in Redis with TTL matching the remaining token lifetime
4. On every request, Gateway checks Redis for the JTI
5. If found, reject the request (token is blacklisted)
6. Redis TTL auto-cleans entries when tokens would have expired naturally

11.2 Advanced Architecture Questions

Q11: How would you scale this security architecture for 10x traffic?

1. Multiple Gateway instances behind a load balancer (JWT is stateless, no affinity needed)
2. Auth Service can be horizontally scaled (DB handles concurrency)
3. Move from symmetric (HMAC) to asymmetric (RSA) keys so services can validate independently
4. Implement token caching in Redis to avoid repeated parsing
5. Use connection pooling for database connections
6. Deploy auth-db as a read replica cluster

Q12: What would you change for a multi-tenant system?

1. Add `tenantId` claim to JWT
2. Add `tenantId` column to user_credentials table
3. Implement tenant isolation at the data access layer
4. Route requests based on tenant ID

5. Separate JWT signing keys per tenant for stronger isolation

Q13: How would you implement OAuth2 with this architecture?

1. Add Spring Authorization Server as a dedicated OAuth2 service
2. Use PKCE (Proof Key for Code Exchange) for public clients
3. Implement authorization_code grant for web, client_credentials for service-to-service
4. Use JWKs (JSON Web Key Sets) for key distribution
5. Replace symmetric JWT signing with asymmetric (RSA-256)

---

12. Future Scalability Considerations

12.1 Migration Path to OAuth2/OIDC

```
CURRENT STATE:
  Custom JWT-based authentication
  HMAC-SHA256 symmetric signing
  Simple RBAC (USER/ADMIN)

FUTURE STATE (Phase 1):
  Spring Authorization Server
  RSA-256 asymmetric signing
  OAuth2 authorization_code + client_credentials
  JWKS endpoint for key distribution

FUTURE STATE (Phase 2):
  OpenID Connect (OIDC)
  Social login (Google, GitHub)
  Multi-factor authentication (MFA)
  Fine-grained permissions (ABAC)

FUTURE STATE (Phase 3):
  External IdP (Keycloak / Okta / Auth0)
  Federated identity
  SAML 2.0 for enterprise SSO
  Zero-trust architecture
```

12.2 Zero Trust Architecture

```
CURRENT: Trust the Gateway, internal services are open
  Client -> Gateway (validate) -> Service (trust headers)

ZERO TRUST: Validate at every layer
  Client -> Gateway (validate JWT)
         -> Service (validate JWT independently)
         -> Database (row-level security)

REQUIRES:
  - Asymmetric keys (public key distributed to all services)
  - mTLS between services (mutual TLS)
```

```
  - Service mesh (Istio / Linkerd) for automatic mTLS
  - Network policies (Kubernetes NetworkPolicy)
```

12.3 Service Mesh Security

```
WITH ISTIO SERVICE MESH:
  - Automatic mTLS between all services
  - No manual JWT forwarding needed
  - AuthorizationPolicy CRDs for fine-grained access
  - RequestAuthentication for JWT validation at mesh level
  - Rate limiting at sidecar level
  - Distributed tracing correlates security events
```

12.4 Observability and Security Monitoring

```
SECURITY MONITORING STACK:
  - Spring Boot Actuator: Health, metrics, info endpoints
  - Micrometer + Prometheus: Auth failure rate metrics
  - Grafana Dashboards: Real-time security monitoring
  - ELK Stack: Centralized security log analysis
  - Kafka Audit Events: Already implemented for admin actions
  - Alerting: Spike in 401s, brute force attempts, unusual IPs
```

Appendix A — Configuration Reference

JWT Configuration (shared between auth-service and api-gateway)

```yaml
jwt:
  secret: YTJiM2M0ZDVlNmY3ZzhoOWkwajFrMmwzbTRuNW82cDdxOHI5czB0MXUydjN3NHg1eTZ6
  expiration: 86400000  # 24 hours in milliseconds
```

Gateway Route Security Matrix

Route ID	Path Pattern	JWT Filter	Circuit Breaker
auth-service	/api/auth/**	NO	YES
user-service	/api/users/, **/admin/users/**	YES	YES
product-service	/api/products/, **/admin/products/**	YES	YES
order-service	/api/orders/, **/admin/orders/**	YES	YES
inventory-service	/api/inventory/, **/admin/inventory/**	YES	YES
payment-service	/api/payments/**	YES	YES
notification-service	/api/notifications/, **/admin/audit-logs/**	YES	YES

Service Security Annotation Usage

Service	@EnableWebSecurity	@EnableMethodSecurity	@PreAuthorize	InternalAuthFilter
Auth Service	YES	NO	NO	NO
User Service	YES	YES	YES	YES
Product Service	YES	YES	YES	YES
Order Service	YES	YES	YES	YES
Inventory Service	YES	YES	YES (class-level)	YES
Notification Service	YES	YES	YES	YES

Appendix B — Glossary

Term	Definition
JWT	JSON Web Token — self-contained, Base64-encoded token with header, payload, and signature
HMAC-SHA256	Hash-based Message Authentication Code using SHA-256 — symmetric signing algorithm
BCrypt	Adaptive password hashing function with configurable cost factor
RBAC	Role-Based Access Control — permissions assigned to roles, roles assigned to users
CSRF	Cross-Site Request Forgery — attack where malicious site tricks browser into making authenticated requests
CORS	Cross-Origin Resource Sharing — mechanism allowing web pages to make requests to different domains
SecurityContext	Spring Security's thread-local storage for the current user's authentication information
OncePerRequestFilter	Spring filter guaranteed to execute exactly once per request
AbstractGatewayFilterFactory	Spring Cloud Gateway's base class for creating custom route filters
SecurityFilterChain	Spring Security's filter chain configuration — defines which endpoints require authentication
@PreAuthorize	Spring Security annotation for method-level authorization using SpEL expressions
GrantedAuthority	Spring Security interface representing a permission (e.g., ROLE_ADMIN)
Bearer Token	OAuth2 token type where possession of the token grants access

mTLS	Mutual TLS — both client and server verify each other's certificates
JWK/JWKS	JSON Web Key / JSON Web Key Set — standardized key format for JWT verification

---

End of Document

This guide provides a comprehensive, enterprise-level reference for building Authentication and Authorization in Spring Boot Microservices. For implementation, start with Step 1 (Database Design) and follow the steps sequentially. Each section is self-contained and can be referenced independently.